Timer_A and Pulse Width Modulation-to-Analog, Analog-to-Digital Conversion

ECE 2564, Embedded Systems

Fall 2025

Overview

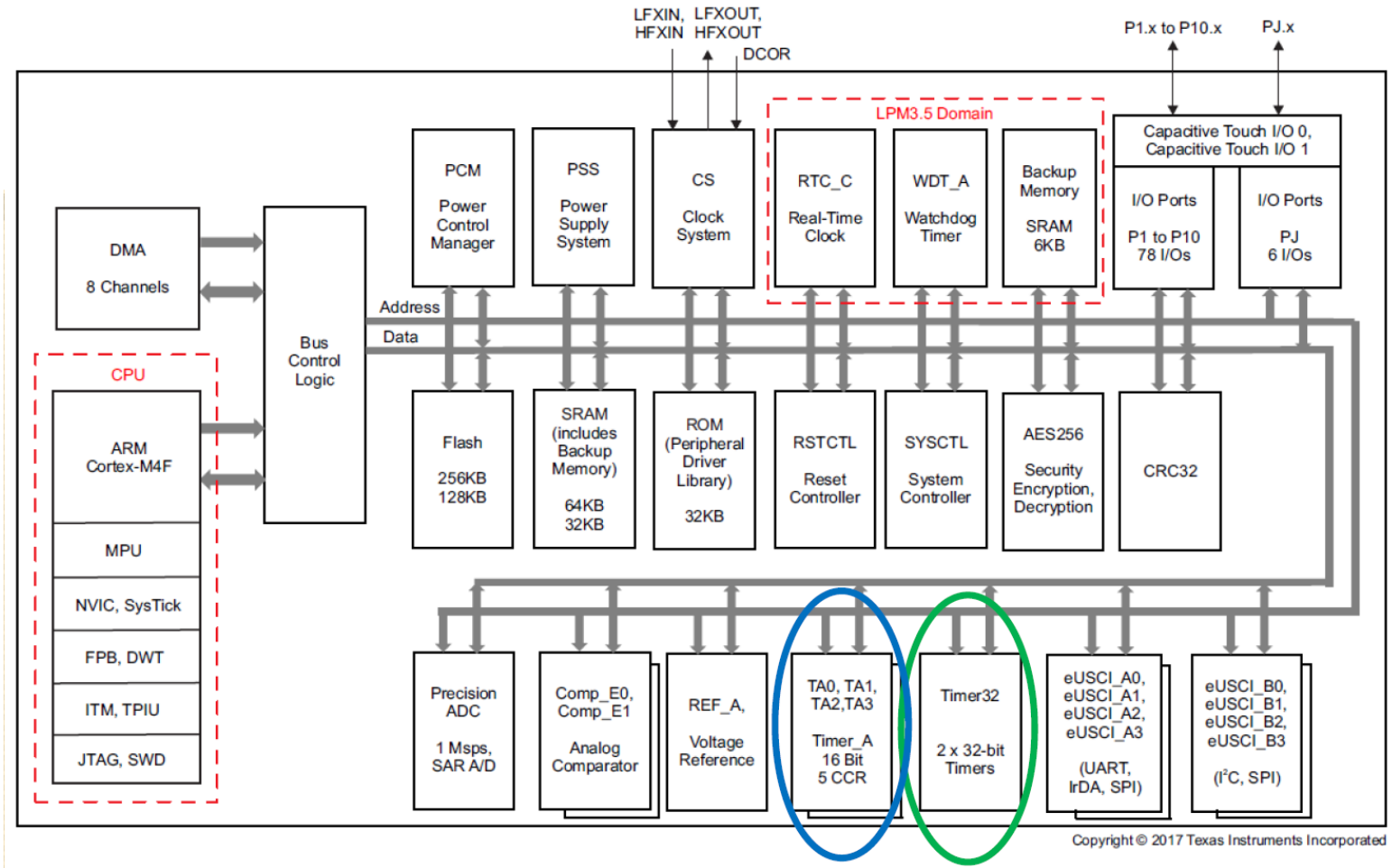
- This module exposes us to Timer_A
- Timer_A is another type of timer supported by the MSP432
- Timer_A is often used to support Pulse Width Modulation (PWM)

Download **this** repository

MSP432 Timers

- We are familiar with the **Timer32** modules on the MSP432
- Now we'll learn about the **Timer_A** modules

The MSP432 Functional Block Diagram: Timers



MSP432 Functional Block Diagram: Highlighting the timer modules

Timer32 Review

- Based on a 32-bit down-counter
- Possible prescale values: $\{1, 16, 256\}$
- Operation modes: Free-running, Periodic, One-shot
- There are two of these on our MSP432 board
  - `TIMER32_0_BASE` and `TIMER32_0_BASE`

Modulation

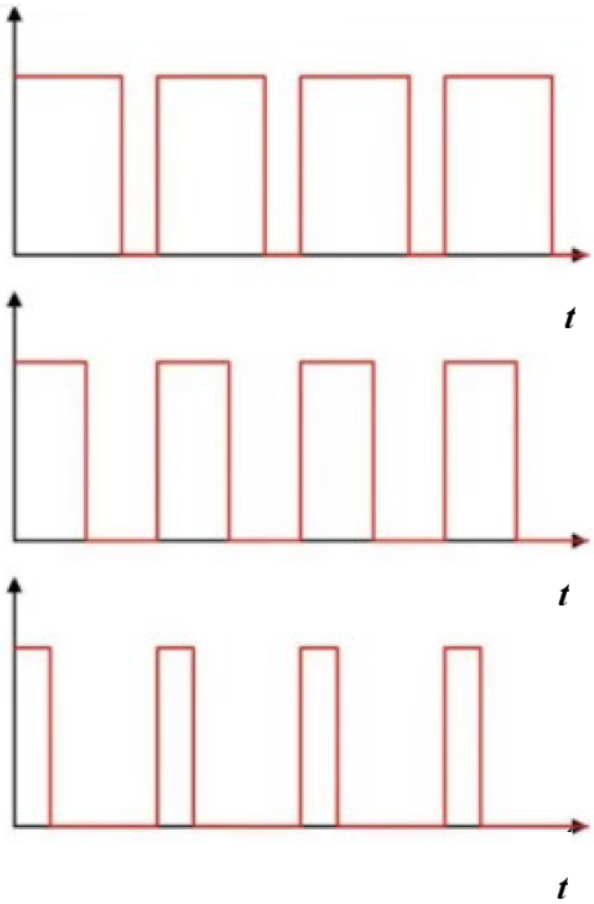
Modulation:

In the electrical engineering domain, *modulation* is the process of varying one or more properties of a periodic waveform

- Amplitude modulation
- Frequency modulation
- Pulse width modulation
  - We vary the width of a periodic pulse
  - This changes the duty cycle, the ratio of the "on" to the period
- PWM Applications
  - Electric heaters with heater elements
  - Servo or stepper motor movement control
  - LED intensity control (dimming the LED)

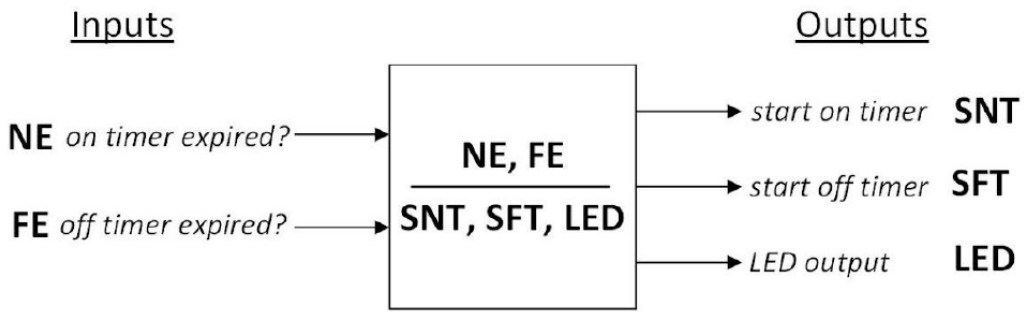
Pulse Width Modulation: Controlling the Duty Cycle

Duty Cycle: the ratio of the "on time" to the period

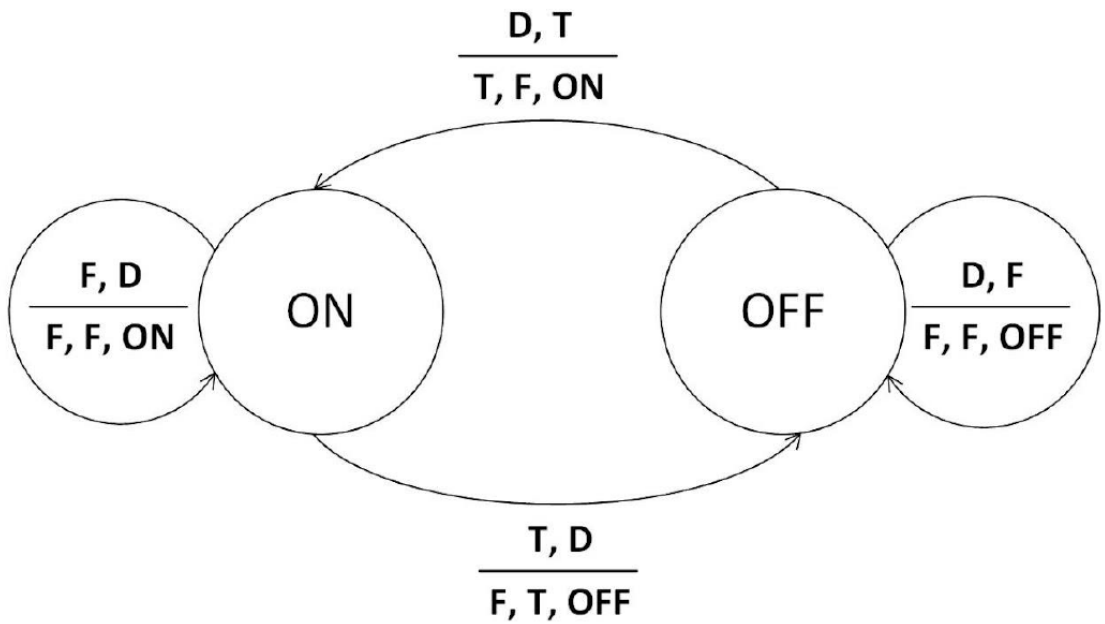


- Do these three signals have the same period?
- Do these three signals have the same duty cycle?
- Which signal has the largest duty cycle?

FSM for Timer32-Based PWM



T = True
F = False
D = Don't Care



The PWM Example Repository Code

- The code is divided into two parts
  - One part uses Timer32 and the other part uses Timer_A
  - The parts are distinguished by a `#ifdef` block and a `#else` block
  - We allow or comment out `#define TIMER32 1` to control the timer mode

- Uncomment line 6 so that `TIMER32` is defined
- Observe the blue LED pulsing
- Observe the blue LED when `DUTY_CYCLE_FRACTION` is varied
- Change `BLOCKING` to 1 on line 10
- Now our blue pulse is not working properly
- What happened?

Pros and Cons of Timer32-Based PWM

- Pros
  - It is easy to implement
  - If you don't have additional hardware times, then this may be the only way to implement PWM
- Cons
  - Variations in the duty cycle will occur if the CPU becomes busy
    - Blocking code will cause the CPU to become busy
  - We are keeping the CPU busy doing doing simple timing tasks
    - Wasting CPU resources

Question: Is it possible to have a timer directly send a signal to the LED (or other peripheral) without going through the CPU?

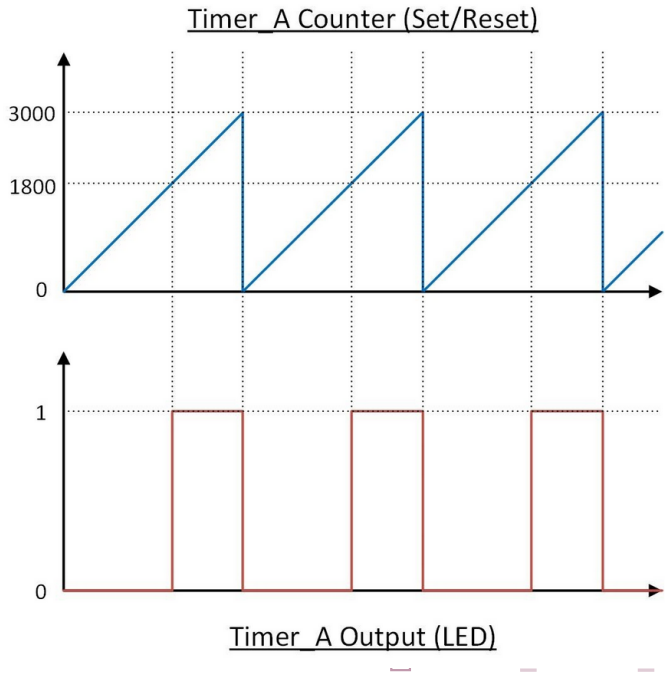
Timer_A Introduction

- Based on a 16-bit up/down counter
- Possible prescale values: $\{1, 2, 4, 6\} \times \{1, 2, 3, 4, 5, 6, 7, 8\}$
- Four operation modes for the counter
- Five "capture/compare blocks"
  - Two modes of *capture* versus *compare*
  - Eight output modes
- There are four of these on our MSP432 board
  - `TIMER_A0_BASE`, ..., `TIMER_A3_BASE`

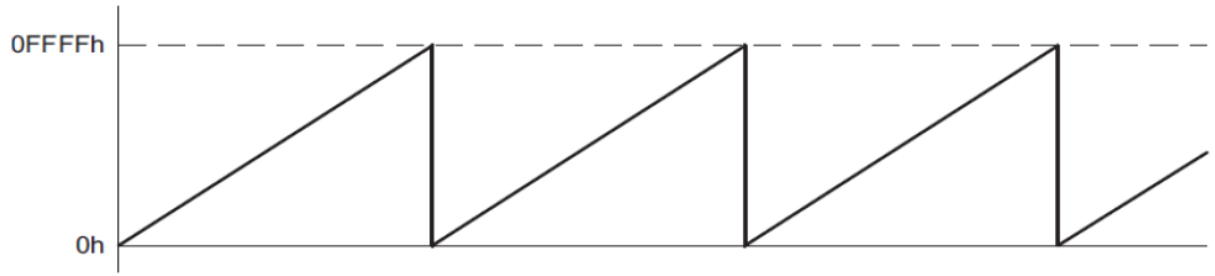
Timer_A: Getting to PWM

- Four Timer_A modules on MSP432
- Each is a 16-bit counter
- Counting modes: up, down, up-down
- Additional registers to hold set/reset values

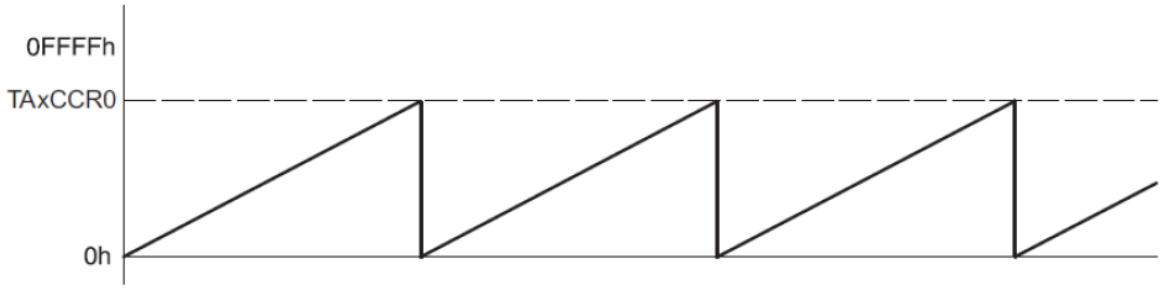
- Example
  - Set: 1800
  - Reset: 3000
  - Duty cycle:
    - $1200/3000 = 40\%$



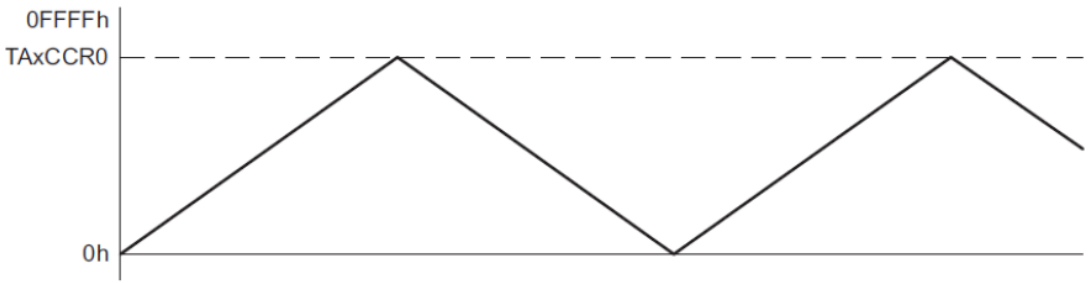
Timer_A Modes



Continuous Mode



Up Mode



Up/Down Mode

Timer Mode Control[1]

Timer Modes

MC	Mode	Description
00	Stop	The timer is halted.
01	Up	The timer repeatedly counts from zero to the value of TAxCCR0
10	Continuous	The timer repeatedly counts from zero to 0FFFFh.
11	Up/down	The timer repeatedly counts from zero up to the value of TAxCCR0 and back down to zero.

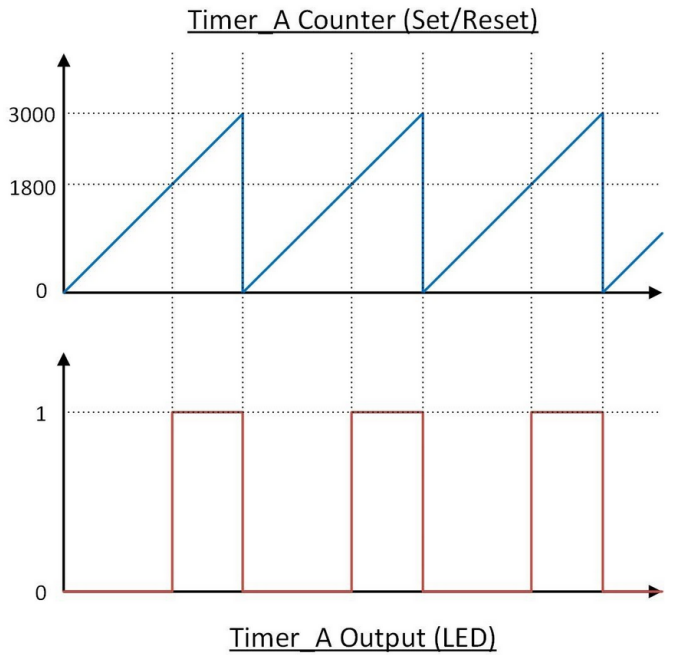
[1] MCU Technical Reference Manual: Table 19-1

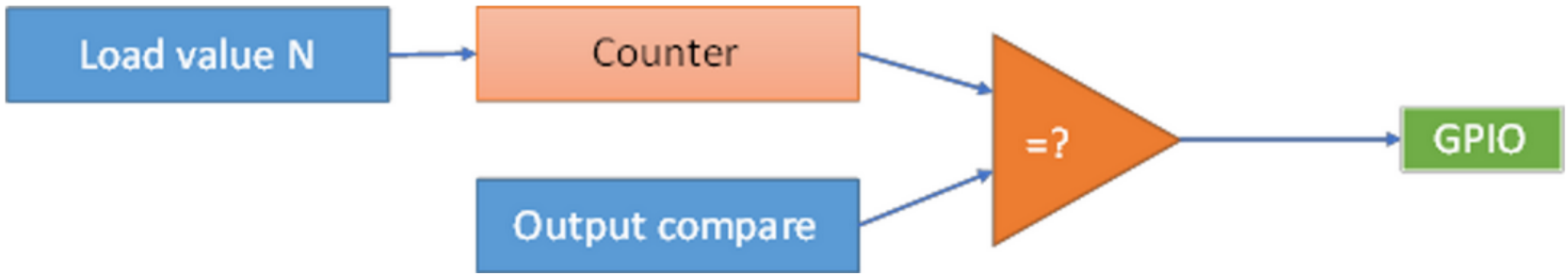
Timer_A Modes

- These modes are also referred to as *count modes* or *operation modes*
- Timer_Ax has *capture/compare* registers named `TAxCCR0`, ..., `TAxCCR3`
- Timer_Ax can be run in these different timer/count/operation modes
  - *Continuous* mode, going from 0 to `0xFFFF` and repeating
  - *Up* mode, going from 0 to TAxCCRn and repeating
  - *Up/down* mode, going from 0 to TAxCCRn, back down to 0 and repeating

Using Timer_A for Output Compare (OC)

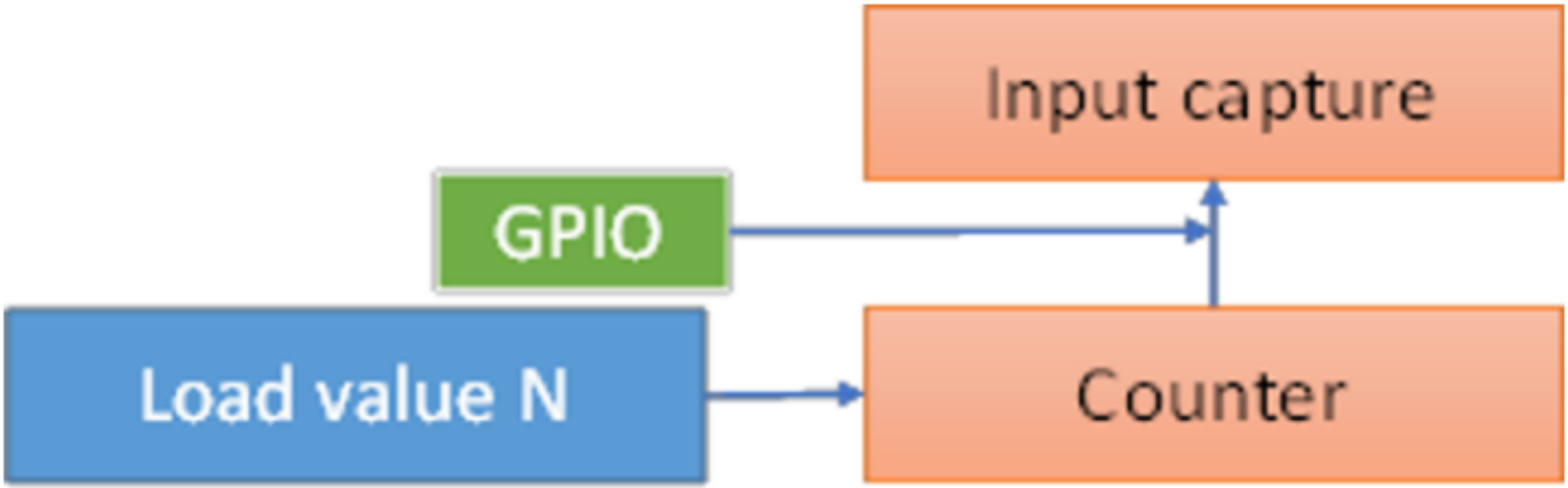
- Load value into Output Compare register
- When the counter value equals the output compare register, trigger the comparator output
  - Example: drive a GPIO signal to 1





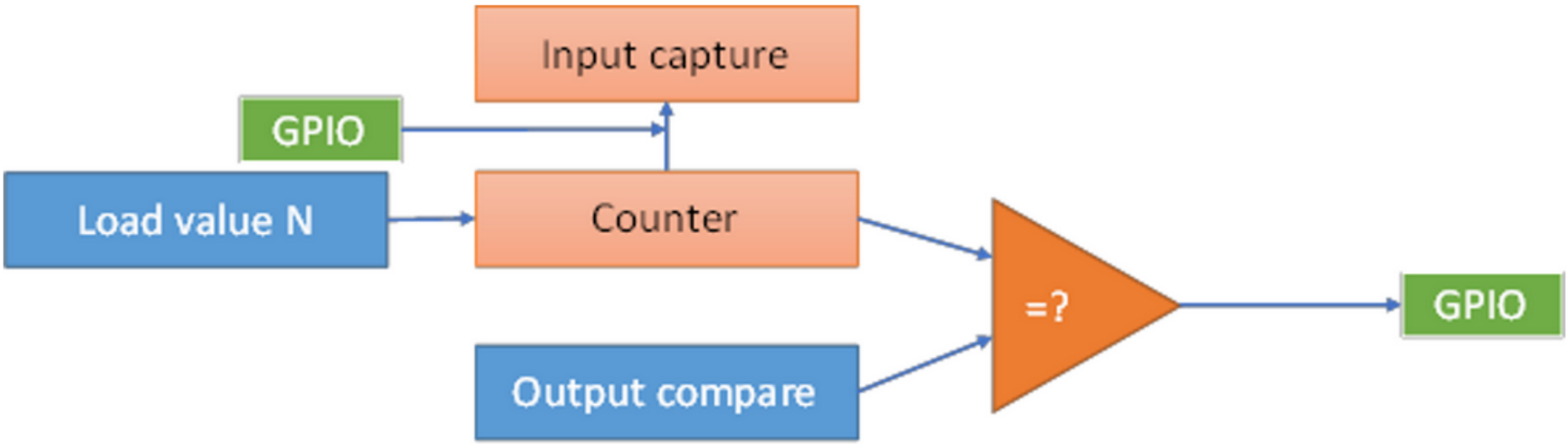
Using Timer_A as Input Capture (IC)

- An external event on a GPIO input pin serves as a *load* signal to of the timer/counter into the Input Capture register
- By calculating the time difference between two captured timer values we can measure the period/frequency/pulse width of an external signal



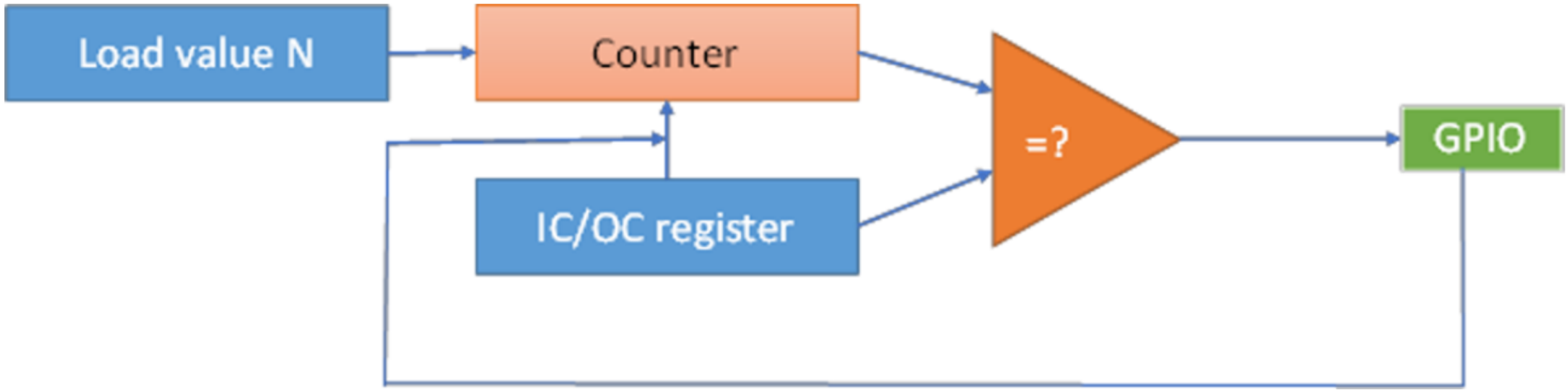
Combined Input Capture/Output Compare

We can combine the diagrams for Input Capture and Output Compare



Alternate Input Capture/Output Compare Diagram

There is just a single register, the IC/OC register, that can be configure in either mode.



Multiple Timer Channels

- Recall: MSP432 has four Timer_As called TA0, . . ., TA3
- Each Timer_A can output to multiple GPIO channels
  - The same counter can be shared

